



Universidad Nacional del Comahue

Facultad de Informática

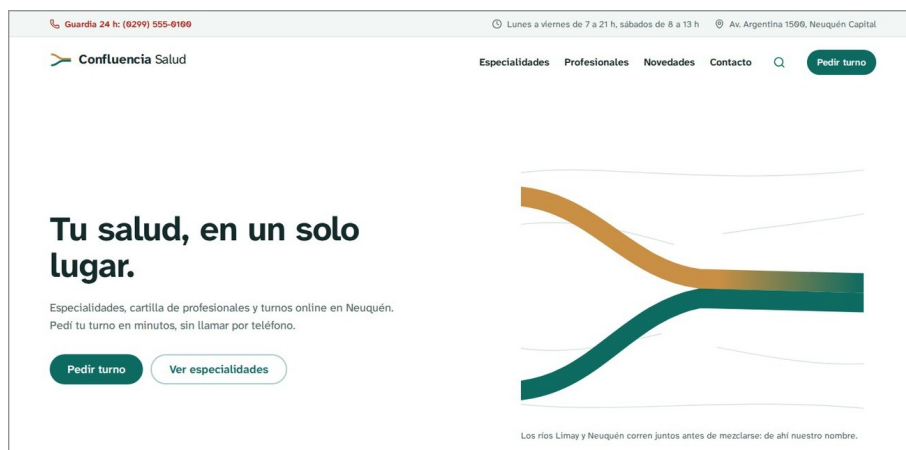
Departamento Ingeniería de Sistemas

Frameworks e Interoperabilidad

Trabajo Práctico N° 2

CMS y Frameworks Web

Confluencia Salud: portal de salud con Next.js y Payload CMS



Integrantes:

Axel Ostrovsky (FAI-4744)

Tomás Sánchez (FAI-4494)

Docentes:

Lara Acuña

Marcos Cruz

Neuquén, octubre de 2026

Índice

1. Introducción.....	3
2. Elección del framework y del CMS.....	3
2.1 Cómo comparamos.....	3
2.2 Framework: Next.js.....	4
2.3 CMS: Payload.....	4
2.4 Cómo funcionan juntos.....	5
3. Template y estilos.....	6
3.1 El template.....	6
3.2 Estilos.....	7
3.3 Lista de cambios.....	8
4. Módulos.....	9
4.1 Usuarios y roles (modificado).....	9
4.2 Cartilla de profesionales (nuevo).....	10
4.3 Novedades de salud (modificado).....	11
4.4 Turnos online (nuevo).....	11
5. Decisiones de implementación y diseño.....	13
5.1 Un solo proyecto con PostgreSQL.....	13
5.2 Validación de los turnos.....	13
5.3 Dos pedidos para el mismo horario.....	14
5.4 Datos personales y fechas.....	14
5.5 Accesibilidad.....	14
5.6 Errores del template y migraciones.....	15
6. Organización del trabajo.....	15
6.1 Tablero.....	15
6.2 Repositorio.....	15
6.3 Pruebas.....	16
7. Conclusiones.....	16
8. Bibliografía.....	16

1. Introducción

Para este trabajo práctico teníamos que elegir un CMS y un framework para el dominio de nuestro Trabajo Final, instalar un template, adaptarlo y crear o modificar al menos tres módulos, sin conectar todavía sistemas entre sí.

Nuestro TF es una plataforma de gestión para centros de salud, hecha en TypeScript con React y Vite en el frontend y NestJS, Prisma y PostgreSQL en el backend. Resuelve la parte interna (organizaciones, usuarios con roles, pacientes y profesionales), pero no tiene un sitio para los pacientes, donde puedan ver las especialidades, buscar un profesional o pedir un turno sin llamar por teléfono. En este TP armamos ese sitio.

Gran parte de lo que se muestra en un sitio así, como los profesionales, los horarios o las notas de prevención, cambia seguido y lo carga el personal administrativo, que no sabe programar, por eso lo manejamos con un CMS. El pedido de turnos, en cambio, tiene reglas propias, y eso lo programamos con el framework.

Al portal lo llamamos Confluencia Salud, porque Neuquén está en la confluencia de los ríos Limay y Neuquén. Tiene cartilla de profesionales, novedades de salud y pedido de turnos online, y se administra desde un panel en español con permisos según el rol de cada usuario. Los datos que aparecen en las capturas (profesionales, matrículas, pacientes y teléfonos) son inventados.

En la sección 2 explicamos cómo elegimos Next.js y Payload. En la 3 mostramos el template antes y después de modificarlo y en la 4, los cuatro módulos. La sección 5 junta las decisiones de implementación y diseño, en la 6 contamos cómo nos organizamos y la 7 tiene las conclusiones.

2. Elección del framework y del CMS

2.1 Cómo comparamos

Usamos los criterios que vimos en la Unidad III. Para el framework tuvimos en cuenta la complejidad del proyecto, el tiempo de aprendizaje, el lenguaje de base, la frecuencia de cambios, el soporte, la posición en el mercado y las funcionalidades, y le agregamos la seguridad. Para el CMS, la arquitectura, el grado de desarrollo, el soporte, la posición en el mercado, la usabilidad, la accesibilidad, la seguridad, la velocidad de descarga y las funcionalidades, y también la licencia y el tiempo de aprendizaje. En los dos casos sumamos un criterio que salía de la consigna, que hubiera un template con front para modificar. Lo otro que pedía, no conectar sistemas entre sí, lo evaluamos en la arquitectura del CMS. Cada criterio lo puntuamos con la escala de la guía práctica, de 1 (malo) a 4 (excelente).

Para la posición en el mercado usamos las descargas semanales de npm entre el 22 y el 28 de septiembre de 2026 [8], y para el grado de desarrollo miramos la última versión publicada de cada uno. Solo comparamos herramientas de JavaScript y TypeScript, porque es lo que usamos en el TF y lo que vimos en PWA con Node.js, Express y PostgreSQL. Cambiar de lenguaje nos iba a llevar tiempo de aprendizaje sin ganar nada para el dominio. NestJS entró en la comparación justamente porque es el backend del TF.

2.2 Framework: Next.js

Next.js sacó el puntaje más alto (Cuadro 1), aunque por un solo punto sobre NestJS. Es un framework de React que renderiza en el servidor, arma las rutas a partir de carpetas, tiene Server Actions para modificar datos [6] y puede generar páginas estáticas que se actualizan solas. Como usa React, lo que ya sabíamos por el frontend del TF nos sirvió directamente. La propia documentación de React recomienda arrancar las aplicaciones nuevas con un framework y pone a Next.js primero en la lista [7]. Lo malo es que cambia muy seguido y mucho de lo que hay en internet es de versiones anteriores. Para que una actualización no nos rompiera el proyecto, dejamos fijas las versiones exactas de las dependencias.

Criterio	Next.js	NestJS	Express
Se ajusta al proyecto (portal, contenidos y turnos)	4	3	2
Lenguaje de base (TypeScript, igual que el TF)	4	4	3
Tiempo de aprendizaje (NestJS ya lo usamos en el TF)	3	4	4
Frecuencia de cambios	2	3	4
Soporte (documentación y comunidad)	4	4	3
Posición en el mercado (descargas semanales en npm)	4 (70 M)	3 (17,5 M)	4 (159 M)
Funcionalidades (rutas, sesiones, idiomas, datos)	3	4	1
Seguridad	3	4	2
Template con front (actividad 2)	4	1	2
Total (sobre 36)	31	30	25

Cuadro 1: Comparación de frameworks. Versiones al 30/09/2026: Next.js 16.3, NestJS 12.1 y Express 5.2.

NestJS quedó cerca porque ya lo conocemos del TF, está en TypeScript y para armar APIs es muy completo, con módulos, guards y validación. Para este TP no nos alcanzaba, porque es solo backend. No trae un template con front ni un CMS, y para hacer el portal tendríamos que haber armado una aplicación de React aparte y conectarla con un CMS por HTTP, que es justo la interoperabilidad que la consigna pedía dejar para más adelante.

Express quedó último. Es un microframework que trae rutas y middleware, y todo lo demás (estructura, validación, vistas y seguridad) hay que hacerlo a mano. Usar React solo tampoco era una opción, porque su propia página lo presenta como una librería para interfaces de usuario [16] y no resuelve rutas, servidor ni datos.

2.3 CMS: Payload

Para el CMS lo que más pesó fue tener el panel y el sitio en una sola aplicación, y de los tres que comparamos eso solo lo cumplía Payload, que terminó con 45 de 48 (Cuadro 2). Es un CMS de código abierto con licencia MIT, escrito en TypeScript, que se instala adentro de una aplicación de Next.js [1]. A partir de la configuración arma el panel de administración, las APIs REST y GraphQL y una Local API que consulta la base directamente, sin pasar por HTTP [1]. Trae autenticación y permisos que se pueden definir por colección, por documento o por campo [3], que es lo que usamos para los roles, y tiene plugins oficiales para formularios, SEO y redirecciones [2]. Perdió un punto en grado de desarrollo y otro en soporte porque la versión 3 sigue cambiando mucho y, aunque se descarga más que Strapi, hay menos tutoriales y respuestas en foros. El tercero lo perdió en tiempo de aprendizaje.

Criterio	Payload	Strapi	Keystone
Arquitectura (CMS y sitio en un solo sistema)	4	2	2
Grado de desarrollo	3	4	3
Soporte	3	4	2
Posición en el mercado (descargas semanales en npm)	4 (1,04 M)	3 (229 K)	1 (15 K)
Usabilidad del panel para personal no técnico	4	4	3
Accesibilidad del sitio que se puede construir	4	3	3
Seguridad (autenticación y permisos)	4	3	4
Velocidad de descarga	4	3	3
Funcionalidades para el dominio	4	4	3
Template con front (actividad 2)	4	2	1
Tiempo de aprendizaje	3	4	3
Licencia	4 (MIT)	3 (MIT y EE)	4 (MIT)
Total (sobre 48)	45	39	32

Cuadro 2: Comparación de CMS. Versiones al 30/09/2026: Payload 3.90, Strapi 5.56 y Keystone 8.1. EE: partes con licencia Enterprise.

A Strapi lo descartamos por la arquitectura. Corre como un servidor aparte, entonces el sitio tendría que ser otra aplicación que consuma su API, o sea, dos aplicaciones que se comunican por HTTP. Tampoco nos convenció que el registro de auditoría (quién cambió qué y cuándo) sea solo del plan Enterprise [10], que no tiene licencia MIT [9], porque en un sistema de salud eso nos parece importante. Keystone tiene el mismo problema porque también funciona como una API GraphQL separada, su proyecto inicial no trae un sitio con front y se descarga mucho menos (unas 15 mil veces por semana, contra más de un millón de Payload [8]).

De la tabla de la consigna miramos Ghost y Apostrophe. Ghost está pensado para blogs y newsletters. Apostrophe sí arma su propio front, pero funciona con MongoDB, y los datos de nuestro dominio (turnos, profesionales y especialidades) son relacionales. Ni Next.js ni Payload aparecen en esa tabla, pero como la consigna dice que “algunas opciones son” esas, los elegimos y los justificamos con estas comparaciones.

2.4 Cómo funcionan juntos

Payload se instala dentro de la carpeta app de Next.js, algo parecido a lo que pasa con Django y Wagtail o con Laravel y OctoberCMS en la tabla de la consigna. Queda una sola aplicación con los grupos de rutas (frontend), que es el sitio público hecho con Next.js, y (payload), que es el panel /admin y las APIs del CMS (Figura 1). Las páginas leen los datos con la Local API, que son simples llamadas a funciones dentro del mismo proceso. Next.js maneja las rutas, el renderizado, la caché y las Server Actions. Lo demás (los datos, el panel, los usuarios y los permisos) lo resuelve Payload, que además tiene hooks, funciones que se ejecutan antes o después de guardar un documento y que usamos bastante.

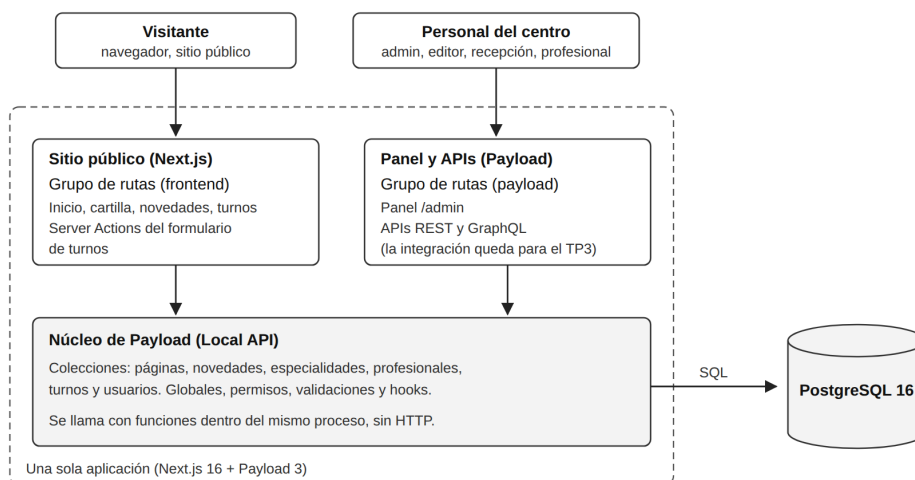


Figura 1: Arquitectura del proyecto.

3. Template y estilos

3.1 El template

Usamos el template *website* oficial de Payload, versión 3.90.2, con licencia MIT [4]. Viene con un sitio hecho en Next.js, Tailwind CSS y shadcn/ui, que tiene un constructor de páginas por bloques, un blog, formularios, búsqueda y modo oscuro. Lo instalamos con PostgreSQL en vez de MongoDB, que es la base que trae por defecto (sección 5.1). Antes de tocar nada lo guardamos en el repositorio con la etiqueta `template-original` y sacamos capturas de pantalla para poder comparar con el resultado (están en la carpeta `docs/capturas/antes`). En las Figuras 2 a 5 se ven algunas pantallas antes y después de los cambios.

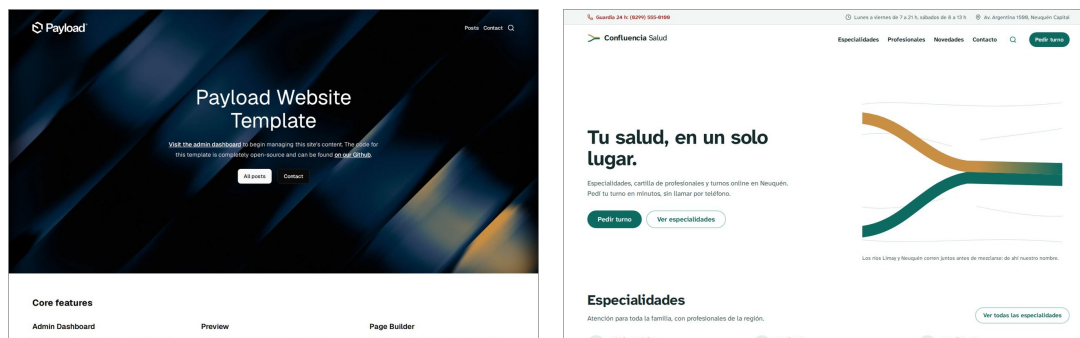


Figura 2: Inicio del template (izquierda) y de Confluencia Salud (derecha).

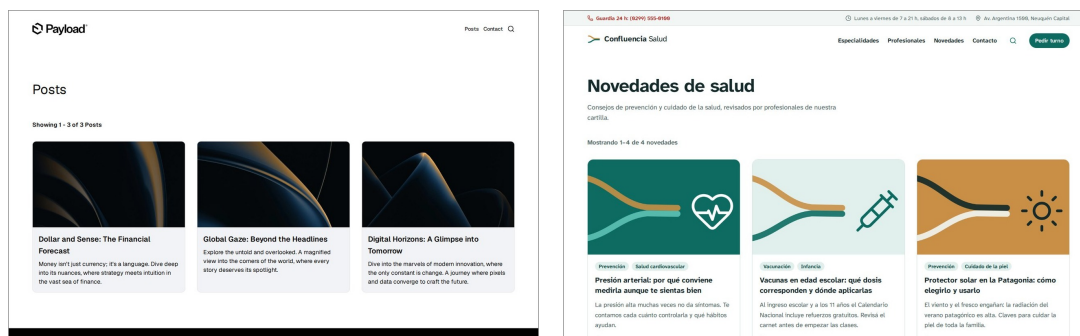


Figura 3: El blog del template y el listado de novedades de salud.

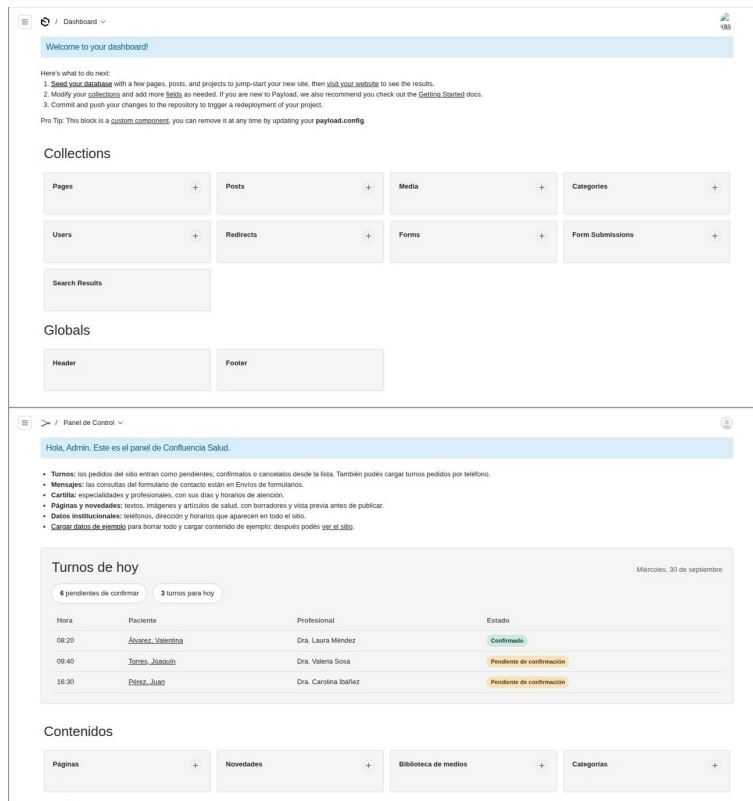


Figura 4: Inicio del panel de administración, antes (arriba) y después (abajo).

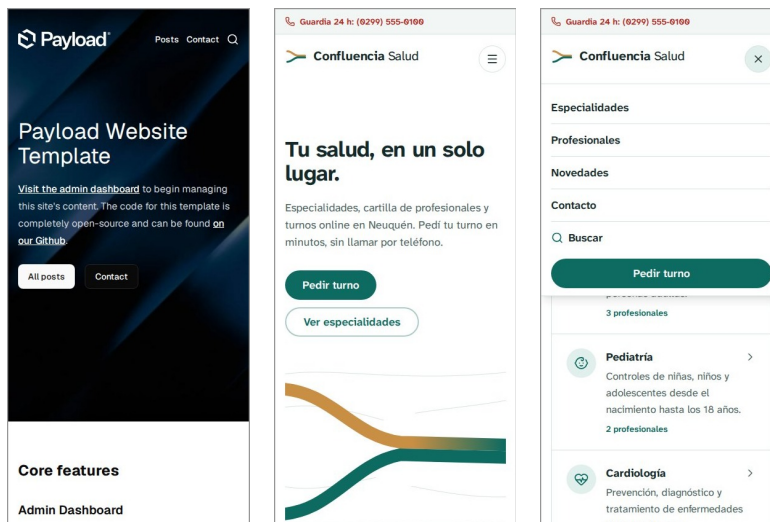


Figura 5: En el celular, el template, nuestra versión y nuestro menú abierto.

3.2 Estilos

El template trae una paleta de grises y la tipografía Geist, que son genéricas y no tienen nada que ver con salud. Para el logo seguimos la idea del nombre y dibujamos dos ríos que se juntan, como el Limay y el Neuquén. Los colores también los sacamos de la zona, como el verde del río y el color de las bardas (Cuadro 3), y no usamos el azul y blanco que tienen casi todas las clínicas.

Color	Modo claro	Modo oscuro	Uso	Contraste
Río (principal)	#0D6B62	#5EC4B6	Botones, enlaces e íconos	6,4:1
Barda (acento)	#C98F45	#DEAA62	Solo logo e ilustraciones	No se usa en texto
Urgencia	#B42318	#FF8A7A	Guardia 24 h y errores	6,6:1
Tinta	#152A2B	#E6F0EE	Texto principal	13,8:1
Texto secundario	#4F6664	#9FB8B4	Descripciones	6,2:1
Niebla	#F2F6F5	#132628	Fondos de tarjetas	-

Cuadro 3: Paleta de colores. El contraste está medido en modo claro (la Tinta sobre Niebla y el resto sobre blanco) y todos los textos superan el mínimo de 4,5:1 de WCAG [12].

Cambiamos la tipografía por Atkinson Hyperlegible Next, que diseñó el Braille Institute pensando en personas con baja visión [13]. Distingue bien letras que se suelen confundir, como la l mayúscula, la l minúscula y el 1. La servimos desde el mismo proyecto (tiene licencia SIL OFL), así no dependemos de Google Fonts y la demo anda sin internet. También subimos el texto base de 16 a 17 px, con un interlineado de 1,6.

Los botones son más redondeados y miden por lo menos 44 px de alto, que es el tamaño que recomienda WCAG para los elementos que se tocan con el dedo [12], y el foco del teclado siempre se ve. La única animación del sitio dibuja los ríos del inicio una vez y no aparece si el usuario tiene activada la opción de reducir el movimiento. El modo oscuro del template lo mantuvimos, con la paleta adaptada.

Por último, traducimos el sitio y el panel al español. Para el panel usamos la internacionalización de Payload [11] y les pusimos nombres propios a las colecciones, los campos y los bloques, pensando en que quien carga los contenidos es personal administrativo.

3.3 Lista de cambios

En el Cuadro 4 juntamos los cambios que le hicimos al template, comparando cómo venía y cómo quedó. El detalle de cada uno, con los archivos que tocamos, está en docs/cambios-template.md.

Elemento	Template original	Nuestra versión
Paleta y tipografía	Grises, Geist, 16 px	Paleta propia con contraste AA, Atkinson Hyperlegible Next, 17 px
Logo, favicon e imagen para redes	Los de Payload	Logo propio en SVG y nombre en texto
Barra superior	No existía	Teléfono de la guardia 24 h, horario y dirección
Encabezado	Enlaces en una fila	Botón "Pedir turno" configurable desde el panel y menú para celulares
Pie de página	Logo y enlaces	Datos de contacto, enlaces, selector de tema y aviso de datos ficticios
Datos de contacto	Escritos en el código	Global "Datos institucionales", editable desde el panel
Portada	Tres variantes con foto	Portada nueva con texto e ilustración de los ríos
Encabezado de nota	Título blanco sobre la foto	Título sobre fondo liso, autor, fecha y tiempo de lectura

Elemento	Template original	Nuestra versión
Tarjetas y botones	Alto variable, "No image", 40 px	Proporción fija, etiquetas, logo si no hay foto, 44 px
Bloques del constructor de páginas	CTA, contenido, imagen, listado y formulario	Se suma "Especialidades destacadas"
Idioma y fechas	Inglés, MM/DD/AAAA	Español en sitio y panel, "12 de agosto de 2026"
URLs	/posts, los slugs pierden las letras con tilde (clínica-médica)	/novedades con redirección, clínica-médica
Accesibilidad	Sin cambios	Enlace "Saltar al contenido", idioma declarado y texto alternativo obligatorio
Panel	Bienvenida para desarrolladores	Bienvenida y menú según el rol, "Turnos de hoy" y logo propio
Contenido de ejemplo	Notas de tecnología con imágenes descargadas	Cartilla, notas de salud, turnos y usuarios de prueba, sin internet

Cuadro 4: Cambios al template.

4. Módulos

La consigna pedía al menos tres módulos e hicimos cuatro (Cuadro 5). Usuarios y Novedades ya venían en el template y los adaptamos, y la Cartilla y los Turnos los hicimos desde cero. En Payload los datos se organizan en colecciones, y cada módulo usa una o más. Con la definición de una colección en TypeScript se generan la tabla en la base, las pantallas del panel, la API y los tipos.

Módulo	Tipo	Para qué sirve
Usuarios y roles	Modificado	Que cada usuario del panel vea y haga solo lo que le toca
Cartilla	Nuevo	Especialidades y profesionales, con sus horarios y obras sociales
Novedades de salud	Modificado	Notas de prevención revisadas por un profesional
Turnos online	Nuevo	Pedir turnos desde el sitio y gestionarlos en el panel

Cuadro 5: Módulos del trabajo.

4.1 Usuarios y roles (modificado)

En el template cualquier usuario logueado puede hacer todo, incluso crear otros usuarios. Le agregamos un campo de roles con cuatro opciones (admin, editor, recepción y profesional) y reescribimos los permisos de todas las colecciones (Cuadro 6). Recepción y profesional son los mismos roles que tiene el TF, y agregamos admin, que puede hacer todo, y editor, que es el que carga los contenidos. Los roles se guardan en el token de la sesión (saveToJWT), así cada control se resuelve sin consultar la base.

La regla más importante es que solo un admin puede cambiar roles. Como es un permiso por campo, nadie puede darse más privilegios editando su propia cuenta. Además, el primer usuario que se crea en una base vacía queda como admin gracias a un hook, y una profesional solo ve sus turnos, porque en su caso la regla de lectura devuelve una condición ("profesional igual a su ficha") que Payload agrega a la consulta [3]. Así, los datos de los otros pacientes ni siquiera salen de la base. El menú del panel también cambia según el rol, pero eso es solo visual, porque los permisos de verdad están en el servidor.

Sección	Admin	Editor	Recepción	Profesional
Páginas, novedades y datos del sitio	Edita	Edita	No	No
Cartilla	Edita	Edita	Consulta	No
Turnos	Todos	No	Todos	Los suyos: estado y notas
Mensajes del formulario de contacto	Lee	No	Lee	No
Usuarios	Todos, asigna roles	Su cuenta	Su cuenta	Su cuenta
Cargar datos de ejemplo	Sí	No	No	No

Cuadro 6: Permisos por rol, definidos en el servidor con las funciones access de Payload.

Igual que en el TF, el usuario y el profesional son cosas distintas. Un profesional puede estar en la cartilla aunque no tenga usuario, y cuando lo tiene, los dos quedan vinculados (Figura 6).

Figura 6: Usuario de una profesional, con su rol y el vínculo con su ficha de la cartilla.

4.2 Cartilla de profesionales (nuevo)

Son dos colecciones nuevas. Especialidades guarda el nombre, un ícono, un resumen, la descripción, el orden y si es destacada. Profesionales guarda el tratamiento, el nombre y el apellido, la matrícula, las especialidades (una relación de muchos a muchos), la foto, las obras sociales y la agenda de atención: los días, el horario de inicio y de fin y cuánto dura cada turno. Esa agenda es la que después usa el módulo de turnos para calcular los horarios libres.

La matrícula tiene que respetar el formato provincial o nacional (por ejemplo MP 4521 o MN 45871), y si alguien escribe “mp4521”, un hook la corrige. También controlamos que la hora de fin sea posterior a la de inicio. Otro hook arma el nombre que se muestra, como “Dra. Laura Méndez”, y cada vez que se guarda un cambio se actualizan las páginas de la cartilla. Los profesionales inactivos no se le muestran al público, así que a una profesional de licencia se la puede ocultar sin borrarla.

Del lado de Next.js hicimos las páginas /especialidades, /especialidades/[slug] y /profesionales, con filtros por especialidad, obra social y nombre o matrícula (Figura 7). El formulario de los filtros usa el método GET y se resuelve en el servidor, por lo que funciona aunque el navegador no ejecute JavaScript y cada búsqueda queda en la URL. Para el constructor de páginas agregamos un bloque nuevo, “Especialidades destacadas”.

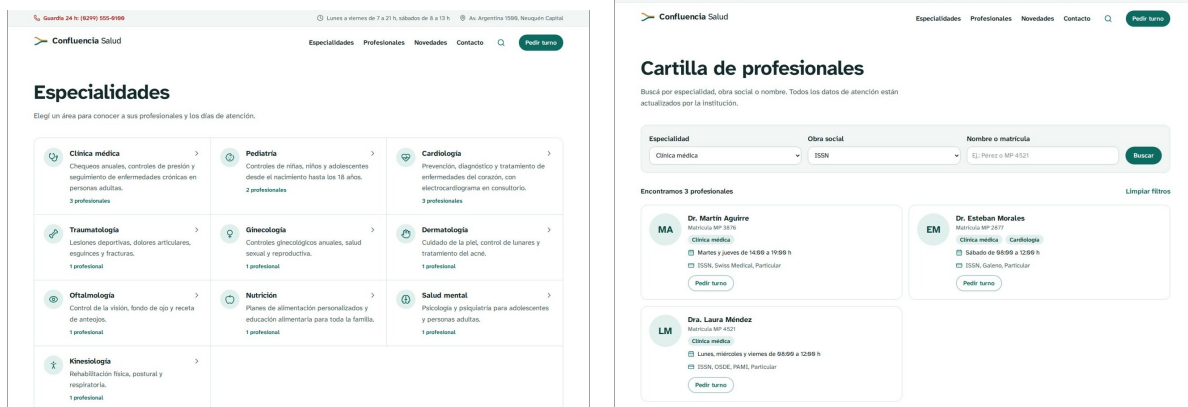


Figura 7: Especialidades y cartilla filtrada por Clínica médica e ISSN.

4.3 Novedades de salud (modificado)

La colección Posts del template la convertimos en Novedades de salud. Además de traducirla, le agregamos un campo “Revisado por”, que es una relación con un profesional de la cartilla. La nota lo muestra con su matrícula y un enlace a su ficha (Figura 8). El tiempo de lectura lo calcula un hook cuando se guarda la nota, a 200 palabras por minuto, y cada nota puede llevar un aviso que aclara que la información es general y no reemplaza una consulta.

Las notas ahora están en /novedades y las direcciones viejas de /posts redirigen a las nuevas; los plugins de SEO y de redirecciones se encargan de las URLs canónicas [2]. También le agregamos listas al editor, que el template no traía. Nos dimos cuenta porque el panel no podía abrir una nota que tenía una lista.

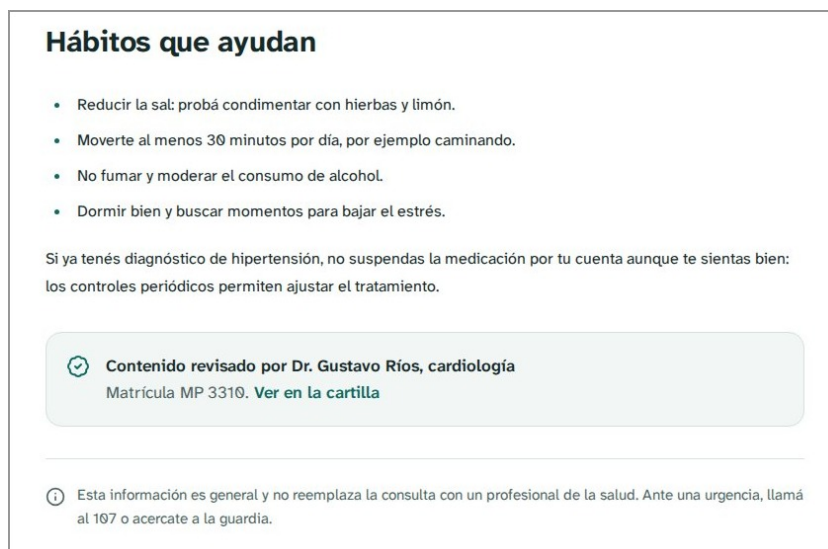


Figura 8: Final de una novedad, con la revisión del profesional y el aviso médico.

4.4 Turnos online (nuevo)

Es el módulo que más trabajo nos llevó, porque usa Next.js y Payload al mismo tiempo. Desde el sitio, cualquiera puede pedir un turno eligiendo la especialidad, el profesional, el día y el horario, y después completando sus datos (Figura 9). El formulario solo muestra los días en que atiende el profesional y, cuando se elige uno, le pide al servidor los horarios libres. Al confirmar, el turno queda pendiente y el paciente recibe un código, por ejemplo CS-T9XVNZ.

En el panel, recepción confirma o cancela los pedidos y carga los turnos que llegan por teléfono (Figura 10), y cada profesional ve su agenda (Figura 11).

Pedir un turno

Elegí la especialidad, el profesional y el horario. Recepción confirma cada pedido por email o teléfono.

1 Especialidad

Clinica médica

2 Profesional

Dr. Martín Aguirre
Matrícula MP 3876
Martes y Jueves de 14:00 a 19:00 h

Dr. Esteban Morales
Matrícula MP 2877
Sábado de 08:00 a 12:00 h

Dr. Laura Méndez
Matrícula MP 4521
Lunes, miércoles y viernes de 08:00 a 12:00 h

3 Día

Viernes, 2 de octubre

Lunes, 5 de octubre

Miércoles, 7 de octubre

Viernes, 9 de octubre

Lunes, 12 de octubre

Miércoles, 14 de octubre

Viernes, 16 de octubre

Lunes, 19 de octubre

Miércoles, 21 de octubre

Viernes, 23 de octubre

Lunes, 26 de octubre

Miércoles, 28 de octubre

Viernes, 30 de octubre

Lunes, 2 de noviembre

Miércoles, 4 de noviembre

Viernes, 6 de noviembre

Lunes, 9 de noviembre

Miércoles, 11 de noviembre

Viernes, 13 de noviembre

4 Horario

08:00

09:20

09:40

10:00

10:20

10:40

11:00

11:20

11:40

No es un canal de urgencias

Los turnos online no son para urgencias. Ante una emergencia llama al 167 o acercate a la guardia.
Guardia: (0299) 555-0100

¿Preferís hablar con alguien?

También podés pedir tu turno por teléfono.
(0299) 555-0200

✓

Recibimos tu pedido de turno

Código de turno

CS-T9XVNZ

Día y hora

Viernes, 2 de octubre, 09:20 h

Profesional

Dr. Laura Méndez (Clínica médica)

Tenemos a escribir a

rocio@example.com

El turno queda pendiente de confirmación. Recepción lo revisa y te confirma por email o por teléfono. Guarda el código por si necesitás cancelarlo.

Pedir otro turno

Volver al inicio

Figura 9: Pedido de un turno. Al costado se aclara que no es un canal de urgencias y se muestra el teléfono de la guardia.

En la parte de Next.js, la página /turnos es un componente de servidor que arma las listas de especialidades y profesionales. El formulario es un componente de cliente que usa las Server Actions consultarHorarios, que devuelve solo las horas libres y nunca datos de otros pacientes, y solicitarTurno, que valida los datos con zod [15] y guarda el pedido. Con useActionState de React 19 mostramos los errores que devuelve el servidor al lado de cada campo, sin que el paciente pierda lo que ya había elegido.

En la parte de Payload, la colección Turnos guarda la especialidad, el profesional, la fecha, la hora, los datos del paciente, el motivo, el estado, el código, de dónde vino el pedido (sitio o panel) y notas internas. Antes de guardar, un hook revisa las reglas, venga el turno del sitio o del panel: que el profesional atienda esa especialidad y ese día, que el horario sea parte de su agenda, que la fecha no haya pasado y que no haya otro turno a esa hora. En el inicio del panel agregamos un componente propio, “Turnos de hoy”, que muestra los turnos del día y los pendientes según los permisos de quien lo mira.

>

/ Turnos

Turnos

Crear

Agenda de turnos. Los pedidos del sitio entran como "Pendiente de confirmación".

Q

Buscar por Código O Resumen

Columnas

Filtros

☐	Resumen	Profesional	Estado	Origen	Código
☐	28/09 08:40 — Díaz, Tomás	Dra. Laura Méndez	Atendido	Sito web	CS-Z8QR88
☐	30/09 08:20 — Álvarez, Valentina	Dra. Laura Méndez	Confirmado	Panel (recepción)	CS-TP4QFN
☐	30/09 09:40 — Torres, Joaquín	Dra. Valeria Sosa	Pendiente de confirmación	Sito web	CS-37KJXG
☐	30/09 16:30 — Pérez, Juan	Dra. Carolina Ibáñez	Pendiente de confirmación	Sito web	CS-RCC9CC
☐	01/10 09:40 — Rodríguez, Carlos	Dra. Valeria Sosa	Confirmado	Sito web	CS-6YU7LQ
☐	01/10 10:20 — Martínez, Ana	Dra. Valeria Sosa	Pendiente de confirmación	Sito web	CS-MW43SR
☐	01/10 10:30 — López, Diego	Dr. Gustavo Ríos	Confirmado	Panel (recepción)	CS-UAQD87
☐	01/10 15:00 — Romero, Sofía	Dra. Julieta Castro	Cancelado	Sito web	CS-KCA8BB
☐	02/10 08:00 — Pérez, Juan	Dra. Laura Méndez	Confirmado	Sito web	CS-HYTMWV
☐	02/10 08:20 — González, María	Dra. Laura Méndez	Pendiente de confirmación	Sito web	CS-SYTM4C

<

>

1

2

1 de 12

Por página: 10

Figura 10: Listado de turnos en el panel, como lo ve recepción.

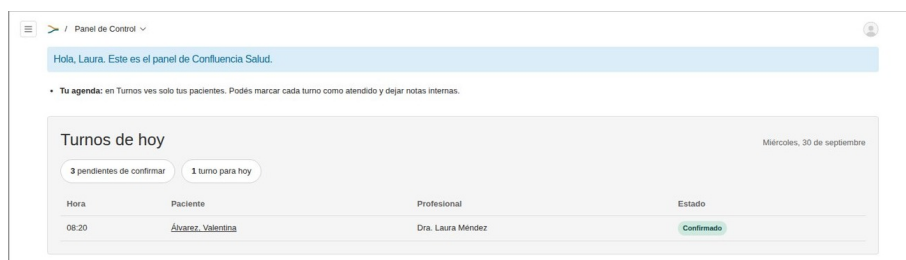


Figura 11: Inicio del panel de una profesional, con sus turnos del día.

Comparados con los del TF, estos turnos son más simples porque se calculan solo con la agenda de cada profesional. En el TF van a depender de los servicios y de la disponibilidad, que todavía no hicimos, pero lo que resolvimos de las reservas duplicadas y de las fechas lo vamos a poder reutilizar.

5. Decisiones de implementación y diseño

En el repositorio dejamos un registro con las 17 decisiones que fuimos tomando (docs/decisiones.md), cada una con su contexto y con lo que descartamos. Aquí resumimos las más importantes.

5.1 Un solo proyecto con PostgreSQL

Montamos el CMS dentro del framework (sección 2.4) para que todo fuera una sola aplicación. También cambiamos MongoDB, que es lo que trae el template, por PostgreSQL con el adaptador oficial de Payload. Los datos del dominio son relacionales y necesitábamos restricciones en la base, como el índice único de los turnos que explicamos en la sección 5.3. Y es la base que usamos en el TF y en la materia PWA.

5.2 Validación de los turnos

Lo que manda el navegador se puede modificar antes de llegar al servidor. La documentación de Next.js avisa que una Server Action (ahí las llaman Server Functions) se puede llamar con un pedido POST directo, sin pasar por la interfaz [6]. Por eso las reglas del turno se controlan en cuatro lugares (Cuadro 7). La API REST de la colección Turnos no deja crear turnos sin una sesión, así que para el público el único camino es la Server Action.

Capa	Dónde	Qué controla
Formulario	Navegador	Muestra solo días de atención y horarios libres. Ayuda al paciente a elegir, pero no sirve como control de seguridad.
Server Action	Servidor (Next.js)	Con zod: el tipo y el formato de cada dato, el consentimiento, que la fecha esté dentro de los próximos 45 días y un campo trampa contra bots.
Hook de la colección	Servidor (Payload)	La especialidad del profesional, el día y el horario de su agenda, que la fecha no haya pasado y que el horario esté libre. También corre para los turnos cargados desde el panel.
Base de datos	PostgreSQL	Índice único sobre profesional, fecha y hora de los turnos vigentes.

Cuadro 7: Dónde se valida un pedido de turno.

5.3 Dos pedidos para el mismo horario

Revisando el hook nos dimos cuenta de un problema. Primero busca si el horario está libre y después guarda, entonces si dos personas mandan el mismo horario al mismo tiempo, las dos búsquedas lo pueden encontrar libre y se guardan los dos turnos. Para evitarlo agregamos un campo oculto con la clave “profesional, fecha y hora” y un índice único en la base, que acepta uno solo de los dos pedidos. En los turnos cancelados esa clave queda vacía (NULL), así el horario se puede volver a reservar. Lo probamos mandando ocho pedidos al mismo tiempo para el mismo horario. Entró uno y los otros siete se rechazaron.

5.4 Datos personales y fechas

Pedimos solo los datos necesarios y el motivo de la consulta es opcional. El formulario pide consentimiento para usar los datos únicamente para gestionar el turno, como indica la Ley 25.326 de Protección de los Datos Personales [14]. Las notas internas no se le muestran al paciente, y el panel no usa Gravatar para no mandarle el email del personal a un servicio externo.

Con las fechas tuvimos que tener cuidado con el huso horario. La base guarda las fechas en UTC, y si un turno del 2 de octubre queda guardado a la medianoche UTC, en Argentina (UTC-3) eso es el 1 de octubre a las 21 h, así que se puede mostrar un día antes. Lo resolvimos guardando la fecha como el día elegido a las 12:00 UTC (las 9 de la mañana acá) y la hora aparte, como texto. Todos los cálculos se hacen en hora de Argentina.

5.5 Accesibilidad

Para revisar la accesibilidad usamos el método Barrier Walkthrough que está en la guía práctica, que analiza las barreras que encuentra cada tipo de usuario. En el Cuadro 8 resumimos qué hicimos para cada uno.

Tipo de usuario	Qué hicimos
Personas ciegas	Texto alternativo obligatorio en las imágenes, etiquetas en todos los campos, títulos ordenados, idioma declarado y errores que el lector de pantalla anuncia.
Baja visión	Tipografía Atkinson Hyperlegible Next, texto de 17 px, contraste AA y zoom sin cortes.
Personas sordas	El sitio no usa audio ni videos y toda la información está en texto.
Daltonismo	La urgencia y los errores no dependen solo del color: también llevan ícono y texto.
Discapacidad motora	Todo se puede usar con teclado, el foco siempre se ve, hay enlace para saltar al contenido, botones de 44 px y opciones de día y horario como botones de radio.
Discapacidad cognitiva	Pasos numerados, un objetivo por pantalla y mensajes de error que dicen cómo corregir.
Sin JavaScript	Páginas generadas en el servidor; los filtros de la cartilla funcionan con un formulario GET.
Epilepsia fotosensible	Una sola animación suave, que se desactiva con la opción de reducir el movimiento.
Motores de búsqueda	Título y descripción en cada página (plugin SEO), mapa del sitio y URLs en castellano.

Cuadro 8: Accesibilidad según los tipos de usuario del Barrier Walkthrough.

5.6 Errores del template y migraciones

Mientras trabajábamos encontramos algunos errores del propio template y los corregimos, cada uno en su commit. El comando `pnpm lint` no funcionaba con la versión 16 de `eslint-config-next`. La paginación calculaba 10 notas por página pero mostraba 12. El teléfono del formulario de contacto era un campo numérico y perdía el cero del principio. La carga de datos de ejemplo, que borra toda la base, la podía ejecutar cualquier usuario logueado. Y al publicar una nota no se actualizaba el inicio.

Para producción, Payload usa migraciones en lugar de sincronizar el esquema solo, como hace en desarrollo [5]. Generamos una migración inicial que se aplica sola al arrancar y probamos la instalación completa desde una base vacía.

6. Organización del trabajo

6.1 Tablero

Organizamos las tareas en un proyecto de Linear que compartimos con los docentes:

<https://linear.app/casdjkojadkasjdjka/project/tp2-cms-y-frameworks-web-3aabda23a78c>

Las 29 tareas están agrupadas en hitos que siguen las partes de la consigna (elección, template y estilos, módulos y entrega). Cada tarea tiene su estado y su prioridad e indica en qué rama de Git se hizo.

6.2 Repositorio

El código está en GitHub:

<https://github.com/axelost2005/tp2-fi-salud>

Trabajamos con una rama por módulo o por arreglo (en cada tarea de Linear anotamos en cuál se hizo) y las fuimos integrando en `develop` con `merge` sin `fast-forward` (`--no-ff`), para que cada módulo se vea en el historial. La rama `main` tiene la versión entregada, con la etiqueta `v1.0.0` (Cuadro 9). En los mensajes de commit ponemos el tipo de cambio (`feat`, `fix`, `docs` o `chore`). Además del código, el repositorio tiene la documentación que pide la consigna: el README con la instalación y el uso, el CHANGELOG, el registro de decisiones, la lista de cambios al template, las pruebas y las capturas de antes y después.

Rama o etiqueta	Qué tiene
template-original (etiqueta)	El template sin modificar, para comparar
feature/tema-salud	Identidad visual, encabezado, pie, portada y traducción
feature/usuarios-roles	Módulo Usuarios y roles
feature/cartilla	Módulo Cartilla y slugs en castellano
feature/novedades	Módulo Novedades de salud
feature/turnos	Módulo Turnos online y contenido de ejemplo
fix/turnos-concurrencia, fix/panel-por-rol	Doble reserva, menú del panel por rol y listas en el editor
fix/tipos-pruebas, fix/pnpm-11	Ajustes para instalarlo y probarlo en otra máquina
chore/migraciones, docs/documentacion	Migraciones, pruebas automáticas y documentación
docs/informe, docs/tablero-linear, docs/entrega	Informe, tablero y datos de la entrega
develop y main (v1.0.0)	Integración y versión entregada

Cuadro 9: Ramas y etiquetas del repositorio.

6.3 Pruebas

Escribimos 33 pruebas automáticas y anotamos las manuales en docs/pruebas.md (Cuadro 10). Mientras las armábamos aparecieron errores que corregimos antes de entregar, como el del editor, que no abría las notas que tenían listas, y el del formulario de turnos, que perdía lo elegido cuando el servidor devolvía un error.

Tipo	Qué prueba	Resultado
Unitarias	Reglas de agenda, fechas en hora de Argentina, código de turno, textos de la cartilla, slugs y rutas	17 de 17
Integración	Con la Local API: validaciones de turnos, doble reserva, cancelación y permisos por rol	9 de 9
Punta a punta	Con Playwright en un navegador: inicio, cartilla, redirección, pedido de turno completo y panel	7 de 7
Manuales	Permisos por la API REST, ocho pedidos simultáneos, instalación desde una base vacía, celular y modo oscuro	15 de 15

Cuadro 10: Pruebas realizadas.

Antes de entregar también lo instalamos desde cero en una máquina virtual con Ubuntu, siguiendo los pasos del README. Ahí nos encontramos con problemas que no esperábamos. La versión 11 de pnpm rechazaba los paquetes porque el reloj de la máquina estaba atrasado once días, y además pedía autorizar de otra forma los paquetes que ejecutan scripts al instalarse. Lo segundo lo corregimos en la rama fix/pnpm-11 y lo primero lo dejamos anotado en el README, por si le pasa a alguien más.

7. Conclusiones

Que Payload se instale adentro de Next.js fue lo que más pesó en la elección (sección 2.4), y creemos que acertamos. El panel, los usuarios, los borradores, los formularios y el SEO ya venían resueltos, así que pudimos dedicarle el tiempo a lo propio de un sitio de salud, como la cartilla y las reglas de los turnos, y tener el sitio y el panel en una sola aplicación, como pedía la consigna.

La parte más difícil fue trabajar con Next.js 16 y Payload 3, que cambian muy seguido y de los que hay poca información actualizada. También aparecieron problemas que no esperábamos, como el huso horario de las fechas, las reservas simultáneas (sección 5.3), los errores del template y la instalación en otra máquina. En el TF vamos a validar los turnos igual que acá, en el servidor y con un índice único en la base, y la accesibilidad la queremos revisar desde el principio.

8. Bibliografía

[1] Payload CMS. What is Payload? Consultado el día 30/09/2026. url:

<https://payloadcms.com/docs/getting-started/what-is-payload>

[2] Payload CMS. Plugins. Consultado el día 30/09/2026. url:

<https://payloadcms.com/docs/plugins/overview>

[3] Payload CMS. Collection Access Control. Consultado el día 30/09/2026. url:

<https://payloadcms.com/docs/access-control/collections>

- [4] Payload CMS. Website Template (repositorio en GitHub). Consultado el día 30/09/2026. url: <https://github.com/payloadcms/payload/tree/main/templates/website>
- [5] Payload CMS. Migrations. Consultado el día 30/09/2026. url: <https://payloadcms.com/docs/database/migrations>
- [6] Next.js. Mutating Data. Consultado el día 30/09/2026. url: <https://nextjs.org/docs/app/getting-started/mutating-data>
- [7] React. Creating a React App. Consultado el día 30/09/2026. url: <https://react.dev/learn/creating-a-react-app>
- [8] npm. Descargas semanales del registro, del 22 al 28/09/2026 (los paquetes que empiezan con @ se consultan de a uno). Consultado el día 30/09/2026. url: <https://api.npmjs.org/downloads/point/2026-09-22:2026-09-28/payload,next,express>
- [9] Strapi. LICENSE (repositorio en GitHub). Consultado el día 30/09/2026. url: <https://github.com/strapi/strapi/blob/develop/LICENSE>
- [10] Strapi. Audit Logs. Consultado el día 30/09/2026. url: <https://docs.strapi.io/cms/features/audit-logs>
- [11] Payload CMS. I18n. Consultado el día 30/09/2026. url: <https://payloadcms.com/docs/configuration/i18n>
- [12] W3C. Web Content Accessibility Guidelines (WCAG) 2.2. Consultado el día 30/09/2026. url: <https://www.w3.org/TR/WCAG22/>
- [13] Braille Institute. Atkinson Hyperlegible Font. Consultado el día 30/09/2026. url: <https://www.brailleinstitute.org/freefont/>
- [14] Argentina.gob.ar. Ley 25.326 de Protección de los Datos Personales. Consultado el día 30/09/2026. url: <https://www.argentina.gob.ar/normativa/nacional/ley-25326-64790>
- [15] Zod. Documentación. Consultado el día 30/09/2026. url: <https://zod.dev>
- [16] React. Página principal. Consultado el día 30/09/2026. url: <https://react.dev>